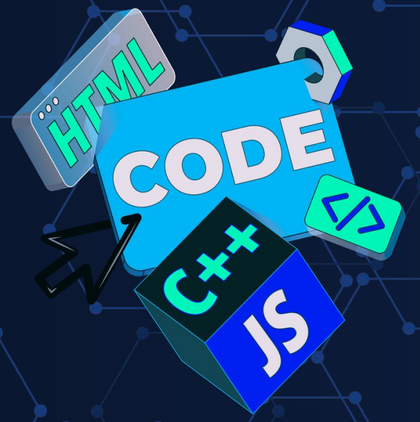
IEEE
SFIT STUDENT BRANCH

SFIT WIE
Women in Engineering

BLITZ

BUILT. LEARN. INNOVATE. TEST. EXECUTE

Enter the Arena of Innovation and Code!



INTRODUCTION

- Team Name: CodeMinds
- Domain Name: Financial Engineering / FinTech
- Problem Statement title: DebtEase: Smart-Settle Debt Minimizer

PROPOSED SOLUTION

- **A detailed explanation of the proposed idea.**

DebtEase is a smart expense-settlement system that takes a group's messy, overlapping debts and calculates the minimum possible number of transactions needed to make everyone financially even. When friends share costs — a trip, rent, meals — they create a web of who-owes-whom that is tedious and confusing to settle manually. DebtEase eliminates that chaos by applying a greedy net-balance algorithm, reducing what could be 12 transfers down to just 4. The result, the reasoning, and the math are all shown transparently — not hidden in a black box.

- **Clearly describe how the solution works.**

The system runs in three deterministic steps:

1. **Compute net balance for every person** —> Total amount paid minus total amount owed. This single number tells you if someone is a creditor or a debtor.
2. **Sort and split** —> Creditors (positive balances) and debtors (negative balances) are separated and sorted by absolute value, largest first.
3. **Greedy match** —> The largest debtor pays the largest creditor the minimum of the two amounts. Repeat until all balances reach zero.

balance[i] = total_paid[i] - total_owed[i] transfer = min(|debtor|, |creditor|)

complexity → O(n log n)

- **Focus on clarity, innovation, and feasibility.**

DebtEase scores on all three judging dimensions:

- **Clarity** — Every settlement decision is explained in plain language. Users see who pays whom, how much, and why. No black box.
- **Innovation — An animated node graph transforms a chaotic debt web into a clean minimal flow in real time. Visual storytelling that competitors don't offer.**
- **Feasibility — The algorithm is ~40 lines of code. Mock API with local fallback ensures zero demo failures. Fully buildable in under 2.5 hours, leaving time for polish.**

- **Explain how the idea solves the selected problem.**

The problem is transaction explosion — **for n people, naive settlement can produce up to n(n-1) payments.** DebtEase **guarantees at most n-1 transactions, the mathematical minimum, by ignoring the original payment paths and working only with net positions .** No one needs to know who paid whom. Only the final net balance matters — and once you have that, the greedy match closes every debt in the fewest possible moves. **A before-vs-after graph visualization shows the tangled web collapsing into clean, minimal arrows**

TECHNICAL APPROACH

Implementation Stack

Python 3.x – core algorithm, backend logic, and API handler

Flask – lightweight REST API server exposing the settlement endpoint

React + D3.js – frontend UI with animated node-graph visualization

NetworkX – graph construction for before/after comparison

Python | Flask | React | D3.js | NetworkX | REST/JSON

Data Handling

Small Predefined Dataset – No Database Required

Format	JSON array of transaction objects in memory.
Sample size	4–8 people, 6–15 raw transactions.
Persistence	Session-scoped, ressts on page reload.
Mock data	3 preset scenarios (trip, rent, dinner)
Validation	Amount > 0, no self-payments, names non-empty.

No DB needed | In-memory state

Algorithmic Logic

Greedy net-balance optimization – O(n log n).

1. **Net Balance per Person**
   balance[i] = ∑ paid - ∑ owed
   Positive = creditor, negative = debtor

2. **Sort Both Lists**
   (Debtor & Creditor) lists 1 by absolute value, largest first. Ignore original paths entirely.

3. **Greedy Match**
   transfer = min(idebtorl, creditor)
   Repeat until all balances = 0.

Complexity O(n log n).
Result ≤ n-1 transactions guaranteed.

No DB needed | In-memory state | 3 demo presets

API Integration

One primary REST API + optional exchange enrichment

POST /apl/settle – Accepts a JSON array of (payer, payee, amount) objects. Returns optimized transactions + step trace JSON
{ settlements[ ], steps[ ], saved_tx }

All responses include a **steps[]** array for full audit trail display in the UI.

POST /api/settle | Audit Steps[] | Optional FX

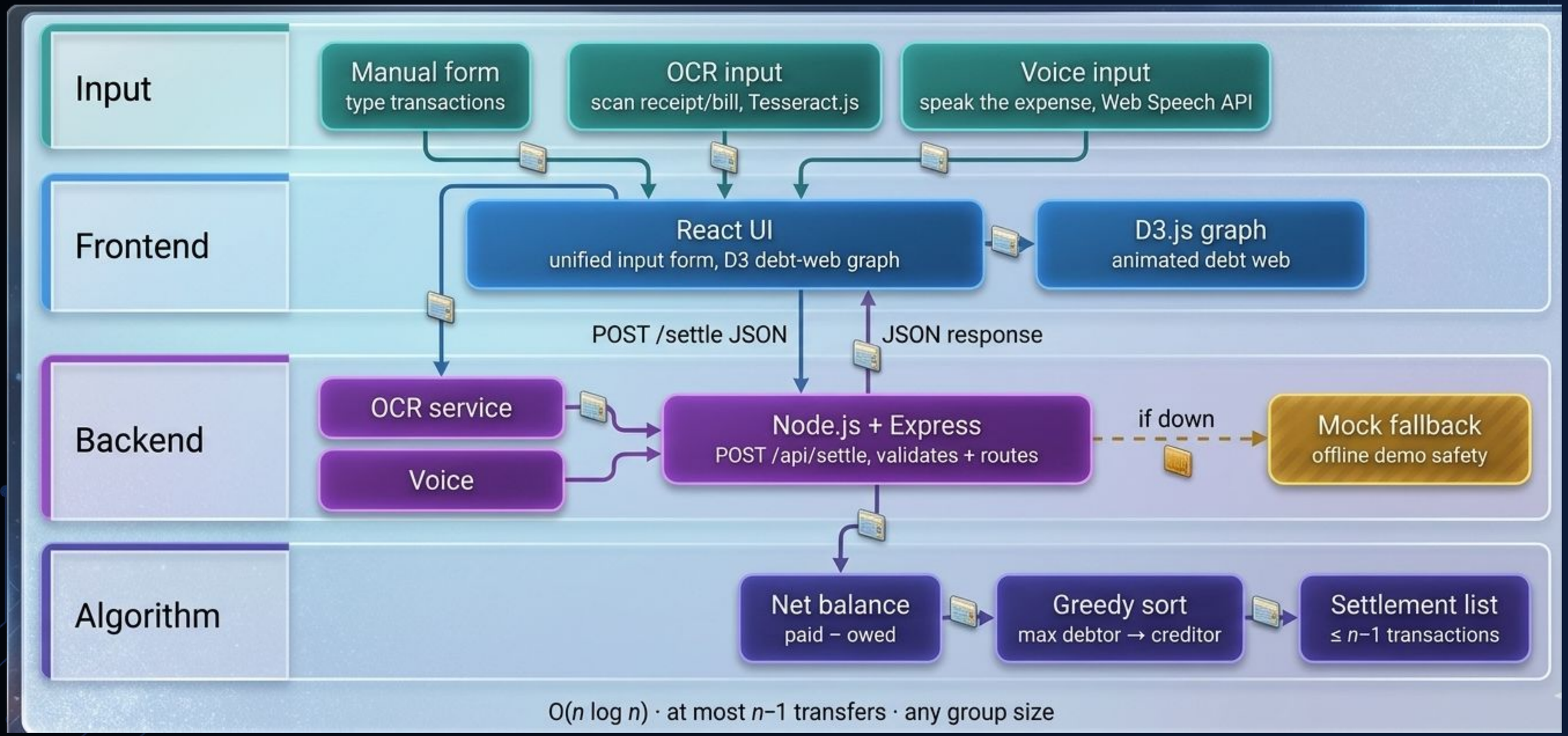
Functional Interface – All Logic Wired End-to-End

- Frontend → Transaction input form -> Add/ remove entries -> Solve POSTs to API.
- Frontend → D3 node graph renders before-vs-after debt web with animated collapse.
- Frontend → Step-by-step audit panel shows each settlement decision in plain language.
- Backend → Flask validates input -> runs gready engine -> returns JSON response.
- Backend → Error handling: malformed input returns 400 with descriptive message.

Fully wired | Animated graph | Audit trail | Mock failback

TECHNICAL APPROACH

Architecture: Overall system design.



KEY FEATURES

MAJOR FEATURES OF THE SOLUTION

NET BALANCE COMPUTATION
Calculates a single position. Determines **Creditors** & **Debtors**. **Original** oran arans.
Original web irrelevant. Final balance matters.

GREEDY OPTIMIZATION ALGORITHM
Guarantees math minimum of **n-1** transactions.
Eliminates complexity.

BEFORE vs AFTER GRAPH VISUALIZATION
Shows dynamic node graph **collapsing** chaotic debt webs to **clean, minimal flows**.

STEP-BY-STEP EXPLAINER PANEL
Plain-language explanations.
Full auditable **transparency**.

MOCK API with LOCAL JSON FALLBACK
Ensures zero demo failures. Seamless automatic **fallback**.

KEY COMPETITIVE INNOVATIONS

VOICE COMMAND EXPENSE ENTRY
Natural language processing. **Real-time** system update.

RECEIPT PHOTO OCR SCANNER
Tesseract.js extracts totals in-browser. **Auto-populates forms.**

"WHAT IF" SIMULATOR
Interactive demos. Immediate **re-optimization** on expense change.

PERSONALITY-BASED DEBT NUDGES
Generates roast-style insights. Shareable, memorable.

MULTI-CURRENCY SUPPORT
Normalizes expenses in **INR, USD, EUR** to a base currency.

SETTLEMENT HISTORY & GROUP MEMORY
Persistent storage across sessions.

REAL-TIME MULTIPLAYER EXPENSE ENTR
Concurrent data entry. **Live, synchronized** graph updates.

HOW THE SYSTEM HANDLES INPUTS AND OUTPUTS

Inputs:
- Expense form (payer, amount, split)
- Mock API / local JSON
- Voice natural language command
- Receipt OCR photo
- Multi-currency

Processing:
- Currency normalization (to INR)
- Net balance calculation
- Creditor/Debtor match
- 'What If' re-reptimization
- Deterministic output logic

Outputs:
- Minimal settlement plan (max n-1)
- Animated visualization graph
- Chaos Score (Before/After)
- Personality nudge (per person)
- WhatsApp summary
- Saved history

Key Functionalities of the Prototype:

- **Greedy loop eliminates one participant per iteration** — guaranteed minimum transactions
- **~40 lines of core algorithm code** — fully buildable within hackathon window
- **API fallback means identical performance with or without internet**
- **Net balance shown before algorithm runs** — users can audit every step
- **Live What If re-optimization** — graph updates the moment an expense changes
- **Fairness index** — shows who consistently over or under-pays
- **Debt forgiveness threshold** — auto-clears micro-debts below ₹10

UI Features for Visualization:

Core: Expense input form · Mock data loader · Before graph panel · After graph panel · Animated morphing transition · Net balance table · Chaos Score card · Settlement steps panel · Transaction count comparison

USP additions: Voice input button · Receipt upload + OCR preview · Currency selector · What If toggle · Personality nudge cards · WhatsApp one-click copy · Settlement history panel · Live multiplayer indicator